

CLI Ticket Manager

Ticket Manager V0.1?

This program is made to track console repair tickets for multiple platforms.

It allows the user to input customer details, select device and multiple repairs.

Each ticket has a status, they are color coded as follows:

- Yellow : Not Started
- Blue: Pending
- Green: Finished

Users can filter by tags or select by ID number which is assigned for each ticket. It is then displayed for the user.

Estimated work time is calculated based on selected repair types.

It is the initial version of a personal tool using minimal libraries that will be expanded upon.

 [GitHub - CaffeinatedAndroid/Repair-Ticket-Manager](https://github.com/CaffeinatedAndroid/Repair-Ticket-Manager)

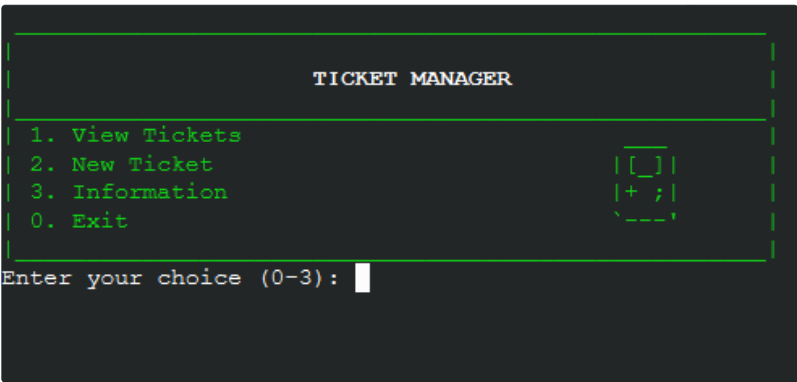
Database?

The database is made up individual tickets store in a sibling folder. Because each ticket is its own object, it prevents accidental corruption of the entire database if tickets are modified incorrectly or missing.

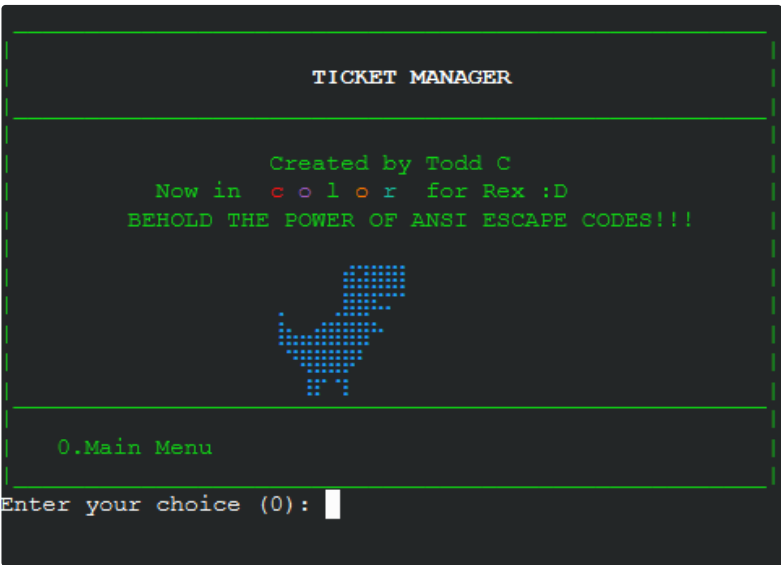
Devices and supported repairs are stored in the Devices.py file for easy repair shop modification and to keep these separate from customer tickets.

How does it work?

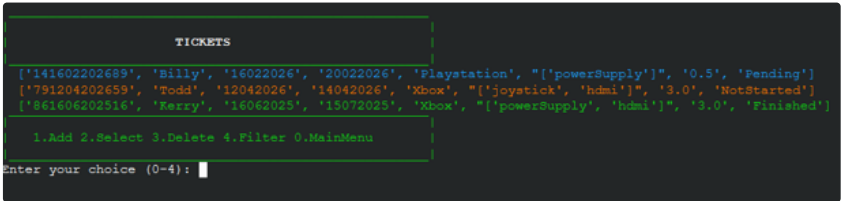
Main menu:



Information Page:



Tickets Page:



New Ticket:

```

NEW TICKET
Name? █

```

Pick Device and repairs:

User can select multiple repairs for the one device, it will calculate total estimated repair time automatically.

```

NEW TICKET
Name? Todd
Drop off date? 13042026
Pick up date? 15042026
Xbox Playstation Switch
Select Device 0-2
Device? 0
Xbox
hdmi joystick powerSupply
Repairs? 1
Selected: joystick
Add more repair items? (1. Yes 2.No): █

```

Ticket Status:

```

NEW TICKET
Name? Todd
Drop off date? 13042026
Pick up date? 15042026
Xbox Playstation Switch
Select Device 0-2
Device? 0
Xbox
hdmi joystick powerSupply
Repairs? 1
Selected: joystick
Add more repair items? (1. Yes 2.No): 2
joystick
Estimated Labour time is: 1.0
0.Pending    1.Finished    2.NotStarted
Status? █

```

Select by ID:

```
TICKETS
[{'141602202689', 'Billy', '16022026', '20022026', 'Playstation', '['powerSupply']", '0.5', 'Pending'}]
[{'791204202659', 'Todd', '12042026', '14042026', 'Xbox', '['joystick', 'hdmi']", '3.0', 'NotStarted'}]
[{'861606202516', 'Kerry', '16062025', '15072025', 'Xbox', '['powerSupply', 'hdmi']", '3.0', 'Finished'}]

1.Add 2.Select 3.Delete 4.Filter 0.MainMenu

Enter your choice (0-4): 2
Enter ticket ID: 141602202689
(0: '141602202689', 1: 'Billy', 2: '16022026', 3: '20022026', 4: 'Playstation', 5: '['powerSupply']", 6: '0.5', 7: 'Pending')
To continue press enter:
```

Select by Term:

```
[{'791204202659', 'Todd', '12042026', '14042026', 'Xbox', '['joystick', 'hdmi']", '3.0', 'NotStarted'}]
[{'861606202516', 'Kerry', '16062025', '15072025', 'Xbox', '['powerSupply', 'hdmi']", '3.0', 'Finished'}]
Press Enter to Continue

1.Add 2.Select 3.Delete 4.Filter 0.MainMenu

Enter your choice (0-4):
```

Delete by ID:

```
TICKETS
[{'141602202689', 'Billy', '16022026', '20022026', 'Playstation', '['powerSupply']", '0.5', 'Pending'}]
[{'791204202659', 'Todd', '12042026', '14042026', 'Xbox', '['joystick', 'hdmi']", '3.0', 'NotStarted'}]
[{'861606202516', 'Kerry', '16062025', '15072025', 'Xbox', '['powerSupply', 'hdmi']", '3.0', 'Finished'}]

1.Add 2.Select 3.Delete 4.Filter 0.MainMenu

Enter your choice (0-4): 3
Enter Ticket ID: 141602202689:
```

Result:

```
TICKETS
[{'791204202659', 'Todd', '12042026', '14042026', 'Xbox', '['joystick', 'hdmi']", '3.0', 'NotStarted'}]
[{'861606202516', 'Kerry', '16062025', '15072025', 'Xbox', '['powerSupply', 'hdmi']", '3.0', 'Finished'}]

1.Add 2.Select 3.Delete 4.Filter 0.MainMenu

Enter your choice (0-4):
```

Code Information?

Main.py is the initial script, it contains majority of menu logic.

Classes.py contains the ticket class and functions for the database.

Repairs.py contains the device and repair dictionary.

Tickets folder is where each ticket is stored.

Future Additions?

I did run low on time in the end, My main additions are to complete the functions i could not finish including:

1. **Export:** Format and save to txt, option to send to printer.
2. **Edit:** allow editing of all fields minus ID, it was half implemented but never was finished. issues revolved around misunderstanding of what dictionary and data i was editing. this can be fixed with refactoring and breaking other functions into more primitive functions that can be easily called internally.
3. **Summary:** Summarize all tickets for range of options "Total Tickets", "Month", "Status", "Hours".

Reflections

I have mixed feelings on my result, I was on track to have all desired functions working however lost steam over the course of the week and other assignments.

If I had more time I would have refactored Classes.py into more manageable chunks rather than cramming everything into the Add function, this would have let me modularize the code in a more readable and intuitive manner. I would also set my naming standard before hand so I don't end up with inconsistency after coming back each day.

Finally I would have reached out and planned earlier rather than the last week, but this is a lesson for next time.